27 FEB 2026

AADK TASK 4

Build an “Interactive Feature Plan” & Implementation Report (Pre-Code + Reflect)

MindMatrix

Feature Idea: Quiz App

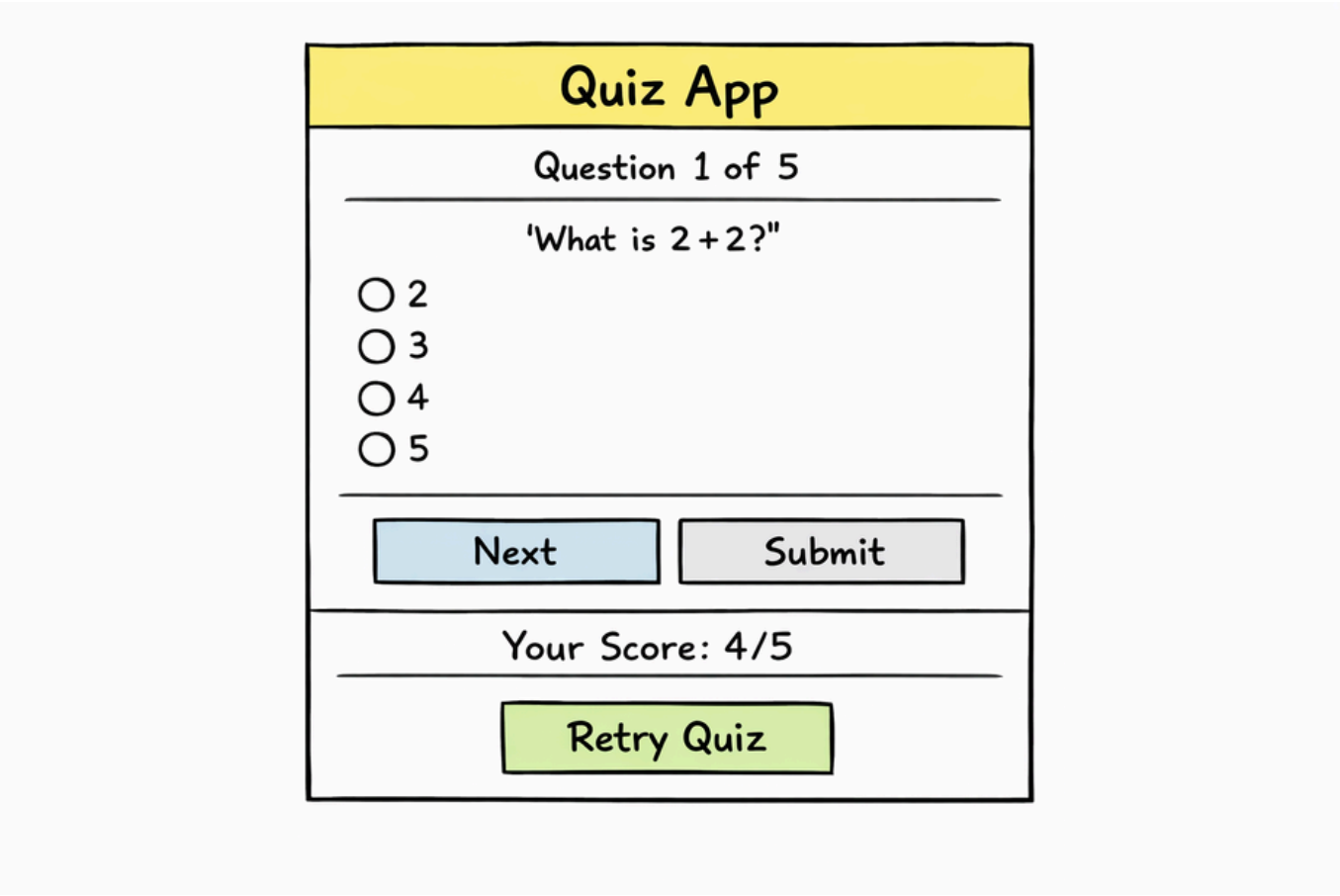
1. Feature Plan

Feature Description

Feature Name: Simple Quiz App
Purpose: Allow users to answer multiple-choice questions and receive a final score.
Target Audience: Students practicing basic knowledge quizzes.
Goal: Provide interactive question navigation, answer validation, and retry option.

The feature includes multiple buttons, state transitions, score tracking, and conditional UI rendering.

2. UI Layout Sketch



3. Button List & Actions

Button 1: Next

- Action: Move to next question.
- State Modified: currentQuestionIndex++
- Condition: Enabled only if an option is selected.

Button 2: Submit

- Action: Calculate final score and show result screen.
- State Modified: quizState = COMPLETED
- Condition: Visible only on last question.

Button 3: Retry Quiz

- Action: Reset quiz to beginning.
- State Modified:

  currentQuestionIndex = 0
  score = 0
  quizState = IN_PROGRESS

Feature 4: Extension Functions

Kotlin Implementation:
Kotlin allows developers to add new functions to existing classes without modifying them.

Java Implementation:
Java does not directly support extension functions.

Advantage:
This makes Kotlin more flexible and keeps code clean and modular.

4. State Model

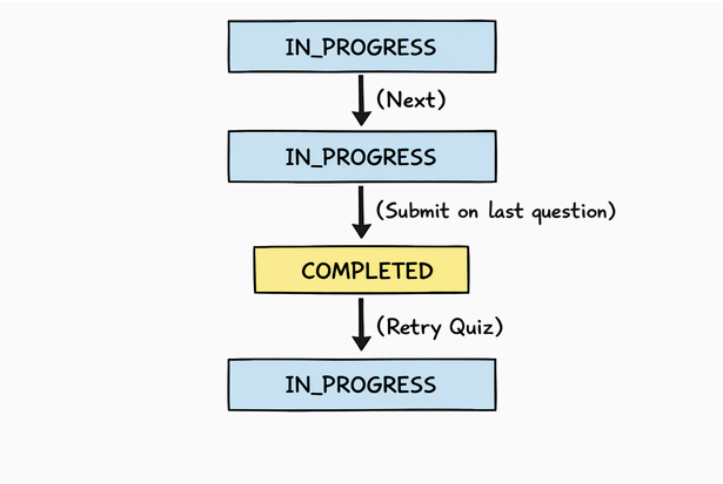
Main States:

State	Description
IN_PROGRESS	User answering questions
COMPLETED	Quiz finished, score displayed

State Transition Table

Current State	Button Pressed	Resulting State
IN_PROGRESS (not last question)	Next	Move to next question
IN_PROGRESS (last question)	Submit	COMPLETED
COMPLETED	Retry Quiz	IN_PROGRESS

State Flow Diagram



5. Test Plan

Test Case 1

- Action: Select answer → Press Next
- Expected Result: Moves to next question, selection resets

Test Case 2

- Action: Complete all questions → Press Submit
- Expected Result: Score screen appears with correct total

Test Case 3

- Action: Press Retry after completion
- Expected Result: Quiz restarts from Question 1, score resets to 0

Test Case 4

- Action: Press Next without selecting an answer
- Expected Result: Button disabled OR show warning message

6. Prototype / Pseudocode

```
var currentQuestionIndex by remember { mutableStateOf(0) }
var selectedAnswer by remember { mutableStateOf<Int?>(null) }
var score by remember { mutableStateOf(0) }
var quizState by remember { mutableStateOf("IN_PROGRESS") }

if (quizState == "IN_PROGRESS") {

  Text("Question ${currentQuestionIndex + 1}")

  Button(
    onClick = {
      if (selectedAnswer == correctAnswer) {
        score++
      }
      currentQuestionIndex++
      selectedAnswer = null
    },
    enabled = selectedAnswer != null   // Advanced case: Disable if no selection
  ) {
    Text("Next")
  }

  if (currentQuestionIndex == lastQuestionIndex) {
    Button(
      onClick = {
        quizState = "COMPLETED"
      }
    ) {
      Text("Submit")
    }
  }
}

if (quizState == "COMPLETED") {

  Text("Your Score: $score")

  Button(
    onClick = {
      currentQuestionIndex = 0
      score = 0
      quizState = "IN_PROGRESS"
    }
  ) {
    Text("Retry Quiz")
  }
}
```

Reflection

One challenge I foresee is managing multiple states correctly without causing inconsistent UI updates. Handling edge cases like fast repeated clicks or navigating beyond the last question may create bugs. I will manage this using clear state variables and conditional UI rendering. To test properly, I will simulate rapid button presses, rotate the screen to test recomposition, and verify that score and question index persist correctly using ViewModel if needed.